

Devoir surveillé terminal

Durée 2h
 (Corrections)

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées.
 Toute réponse doit être justifiée.*

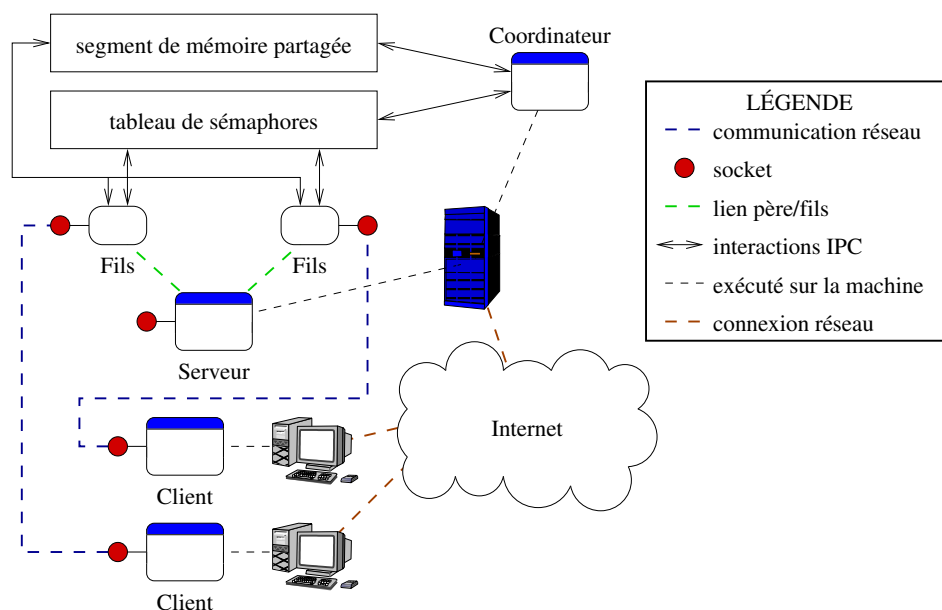
Nous souhaitons développer un jeu de bataille navale en temps réel, multi-joueurs et en réseau. L'application répartie est constituée de trois applications distinctes : un *coordinateur*, un *client* et un *serveur*.

Le but du jeu. Chaque joueur possède un nombre de bateaux placés sur un plateau de jeu qui est une grille avec une largeur et une hauteur données : chaque case peut contenir plusieurs bateaux. Le *client* est l'application exécutée par chaque joueur. Il affiche les cases du plateau de jeu, les bateaux du joueur et il permet au joueur de réaliser des actions. Au début du jeu, le joueur place ses bateaux sur le plateau de jeu et attend le début de la partie. Celle-ci démarre une fois que le nombre de joueurs est atteint et qu'ils sont tous prêts. Le jeu se déroule en tour par tour : à chaque tour, tous les joueurs peuvent réaliser simultanément une action qui peut être soit un tir, soit un déplacement de bateau. Pour tirer, le joueur choisit une case : si un ou plusieurs bateaux sont présents, il sont détruits. Pour déplacer un bateau, le joueur en sélectionne un puis clique sur une case adjacente (haut, bas, droite ou gauche). Un joueur a perdu lorsqu'il ne possède plus de bateau. Le jeu s'arrête lorsqu'il ne reste plus qu'un seul joueur.

Les applications. Le *serveur* permet aux *clients* de se connecter au jeu depuis des machines distantes. Nous utiliserons les communications en mode connecté. À chaque connexion d'un *client*, le *serveur* crée un processus fils. Ce dernier récupère les informations sur le jeu depuis un segment de mémoire partagée (SMP). Un tableau de sémaphores (TdS) est utilisé pour la synchronisation. Le but du *coordinateur*, exécuté sur la même machine, est d'initialiser le jeu. En paramètre, il prend la largeur et la hauteur du plateau de jeu, ainsi que le nombre de joueurs et le nombre de bateaux pour chaque joueur. Il crée le SMP et le TdS.

1°) **Généralités (1 point).** Représentez l'architecture de l'application en précisant les interactions entre les différentes applications, les outils IPC et en représentant les communications réseau.

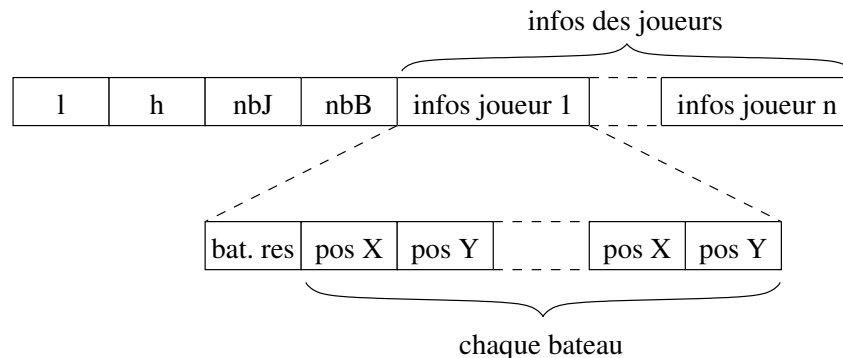
Solution : Voici une architecture possible :



2°) **Le segment de mémoire partagée (8 points).** Le SMP contient toutes les informations partagées entre les joueurs et est accessible par les fils du serveur. Les données sont la configuration du jeu (dimensions, nombre de joueurs et de bateaux) et les informations sur chaque joueur (le nombre de bateaux restant et les positions des bateaux).

a) Donnez le contenu du SMP sous la forme d'un schéma ; expliquez l'intérêt de chaque donnée, le type utilisé et la taille.

Solution : (1 point). Le segment doit contenir la largeur et la hauteur du plateau (deux entiers), le nombre de joueurs et de bateaux (deux entiers). Pour chaque joueur, nous avons le nombre de bateaux restants (un entier), les positions de chaque bateau (deux entiers). Si un bateau est coulé, on lui met la position (-1, -1), par exemple. Voici un schéma possible :



b) Pour pouvoir consulter ou modifier les données du SMP, nous souhaitons utiliser une structure en C nommée `segment_t`. Donnez cette structure et les autres structures dont vous auriez éventuellement besoin.

Solution : (1 point). Voici les structures nécessaires :

```
/* Position d'un bateau */
typedef struct {
    int x;
    int y;
} position_t;

/* Données d'un joueur */
typedef struct {
    int *bateauxRestant;
    position_t *posBateaux;
} joueur_t;

/* Structure du segment */
typedef struct {
    int *largeur;
    int *hauteur;
    int *nbJoueurs;
    int *nbBateaux;
    joueur_t *joueurs;
} segment_t;
```

La fonction `creer_segment` crée le SMP et retourne une structure `segment_t` permettant de le mapper. Ses paramètres sont : `cle` est la clé IPC du segment, `l` et `h` sont respectivement la largeur et la hauteur de la grille, `nbJoueurs` est le nombre de joueurs et `nbBateaux` est le nombre de bateaux de chaque joueur. La signature est la suivante :

```
segment_t creer_segment(key_t cle, int l, int h, int nbJoueurs, int nbBateaux)
```

c) Expliquez les actions réalisées par cette fonction.

Solution : (1 point).

d) Donnez le code de la fonction `creer_segment`.

Solution : (4 points). Le code peut être le suivant :

```
segment_t creer_segment(key_t cle, int l, int h, int nbJoueurs, int nbBateaux) {
    void *adr;
    segment_t resultat;
    size_t taille = sizeof(int) * 4 + (sizeof(int) + sizeof(int) * 2 * nbBateaux) *
        nbJoueurs;

    adr = shmget(cle, taille, IPC_CREAT | S_IRUSR | S_IWUSR);
    resultat.largeur = &adr[0];
    resultat.hauteur = &adr[1];
    resultat.nbJoueurs = &adr[2];
    resultat.nbBateaux = &adr[3];

    resultat->largeur = l;
    resultat->hauteur = h;
    resultat->nbJoueurs = nbJoueurs;
    resultat->nbBateaux = nbBateaux;

    resultat.joueurs = (joueur_t*)malloc(sizeof(joueur_t) * resultat->nbJoueurs);
    courant = &adr[4];
    for(i = 0; i < nbJoueurs; i++) {
        resultat.joueurs[i].bateauRestant = courant;
        resultat->joueurs[i]->bateauxRestant = nbBateaux;
        resultat.joueurs[i].posBateaux = (position_t*)&courant[1];
        resultat->joueurs[i]->posBateaux[j].x = 0;
        resultat->joueurs[i]->posBateaux[j].y = 0;
        courant += 1 + resultat.nbBateaux * 2;
    }
    return resultat;
}
```

3°) **Les sockets** (5 points). Les communications entre les *clients* et les fils du *serveur* sont en mode connecté. Le *serveur* écoute sur un numéro de port spécifié dans l'application par la constante `PORT_ECOUTE`.

a) Donnez le code principal du serveur permettant de créer la socket et de créer un fils pour chaque connexion. Nous ne nous intéresserons pas à l'arrêt du serveur qui tournera en boucle. Le code du fils est situé dans une fonction `void fils(...)` dont vous donnerez uniquement la signature.

Solution : (2 points). Voici le code :

```
void fils(int fd);

int main() {
    int fd;
    sockaddr_in adresse;

    fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    memset(&adresse, 0, sizeof(struct sockaddr_in));
    adresse.sin_family = AF_INET;
    adresse.sin_addr.s_addr = htonl(INADDR_ANY);
    adresse.sin_port = htons(PORT_ECOUTE);
    bind(fd, (struct sockaddr*)&adresse, sizeof(struct sockaddr_in));

    listen(fd, 1024);
    while(1) {
        sockclient = accept(fd, NULL, NULL);
        if(fork() == 0) {
            close(fd);
```

```

        fils(sockclient);
    }
    close(sockclient);
}
return EXIT_SUCCESS;
}

```

b) En supposant que l'adresse IP de la machine sur laquelle s'exécute le *serveur* est spécifiée en argument du programme *client*, donnez la portion de code du *client* permettant d'établir une connexion avec le *serveur*.

Solution : (1 point). Voici un exemple de code :

```

int main(int argc, char *argv[]) {
    int fd;
    sockaddr_in adresse;

    fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

    memset(&adresse, 0, sizeof(struct sockaddr_in));
    adresse.sin_family = AF_INET;
    inet_pton(AF_INET, argv[1], &adresse.sin_addr.s_addr);
    adresse.sin_port = htons(PORT_ECOUTE);

    connect(fd, (struct sockaddr*)&adresse, sizeof(adresse));
    ...
}

```

c) Combien de numéro de port sont utilisés sur le serveur ? Combien de sockets ? À quels endroits doit-on faire des appels à `close` ?

Solution : (1 point). Un seul numéro de port est nécessaire. Une socket de connexion est utilisée sur le serveur et une socket de communication est créée à chaque connexion. Elle est transmise aux fils et fermée immédiatement dans le père. Il faut faire un `close` après la boucle `while` (à l'arrêt du serveur), dans chaque fils, on ferme la socket de connexion et dans le père, on ferme immédiatement la socket de communication. À la fin de la fonction du fils, on ferme la socket de communication.

d) Nous souhaitons que le serveur s'arrête dès qu'il reçoit `CRTL + C` et qu'il se mette en attente de la fin de ses fils. Expliquez quels mécanismes doivent être mis en place et comment modifier votre code précédent.

Solution : (1 point). Les touches `CRTL + C` produisent un signal `SIGINT`. Il faut mettre en place un gestionnaire (à l'aide de `sigaction`) qui place une variable globale `stop` à 1. La boucle `while` s'arrête dès que la variable `stop` est à 1. Attention à l'appel `accept` qui sera interrompu (retour à -1 et `errno` à `EINTR`). Pour attendre la fin des fils, on fait autant de `wait` que nécessaire. On peut placer un compteur à chaque création de fils. Voici les modification de code (non demandées) :

```

int stop = 0;

void gestionnaire(int signum) {
    stop = 1;
}

int main() {
    ...
    int nbClients, i;
    struct sigaction action;

    action.sa_sigaction = gestionnaire;
    sigemptyset(&action.sa_mask);
    action.sa_flags = SA_HANDLER;
    sigaction(SIGINT, &action, NULL);
    ...
}

```

```

while (stop == 0) {
    sockclient = accept(fd, (struct sockaddr*)&adresseClient, &taille);
    if (sockclient != -1) {
        nbClients++;
        ...
    }
}
close(fd);

for (i = 0; i < nbClients; i++) wait(NULL);

return EXIT_SUCCESS;
}

```

4°) **Le tableau de sémaphores** (6 points). Le *coordinateur* crée le TdS qui sera utilisé pour les différentes synchronisations entre les applications.

a) Nous souhaitons utiliser un seul sémaphore pour vérifier que tous les joueurs attendus sont bien connectés et s'assurer qu'il n'y ait pas plus du nombre de joueurs autorisés. Expliquez les différentes actions à réaliser (initialisation, opérations généralisées, etc.) par le *coordinateur* et les fils du serveur.

Solution : (1 point). Un sémaphore (S_1) est créé par le *coordinateur* et initialisé à nb qui est le nombre de joueurs autorisé. Le *coordinateur* attend que S_1 soit à 0 : $ATT(S_1)$. L'idée est de décrémenter S_1 à chaque nouveau joueur et dès qu'il atteint 0, c'est que le nombre de joueurs est atteint. Chaque fils fait donc un $P(S_1)$ puis un $ATT(S_1)$. Les clients suivants seront bloqués : on peut donc faire un $P(S_1)$ non bloquant et s'il échoue, c'est que le nombre de clients est déjà atteint.

b) Avec votre solution, que se passe-t-il si un joueur tente de se connecter alors que le nombre maximum de joueurs est déjà atteint ? Si ce n'est pas déjà fait, proposez une solution pour que le fils du *serveur* s'arrête correctement.

Solution : (0,5 point). Les clients suivants seront bloqués : on peut donc faire un $P(S_1)$ non bloquant et s'il échoue, c'est que le nombre de clients est déjà atteint.

c) Pour éviter de créer un fils inutilement si le nombre de joueurs est atteint, proposez une solution au niveau du *serveur*.

Solution : (0,5 point). Avec un simple compteur : dès qu'un client se connecte, on l'incrémente de 1. Dès que le nombre de clients est atteint, on coupe la socket de connexion. Le serveur se met alors en attente de la fin de ses fils.

d) Un tour s'arrête dès que tous les joueurs ont réalisé leur action (soit un tir, soit un déplacement). En utilisant un ou plusieurs sémaphores, expliquez comment gérer les tours : un seul coup par joueur, passer au tour suivant, etc. Vous préciserez les sémaphore(s) utilisé(s), leur initialisation, les opérations généralisées, etc.

Solution : (1 point). Il faut que tous les clients soient bloqués tant que le tour n'a pas commencé (ou n'est pas terminé). On peut utiliser le même principe que précédemment. Le *coordinateur* crée un sémaphore S_2 et l'initialise avec le nombre de clients. Chaque fils fait un $P(S_2, 1)$ après une action puis un $ATT(S_2)$. Le *coordinateur* fait en boucle un $[ATT(S_2), V(S_2, nb)]$ avec nb le nombre de joueurs.

e) Les actions sont traitées immédiatement, pas uniquement à la fin du tour. Expliquez les problèmes de synchronisation que cela pose. Proposez une solution en utilisant un nombre minimal de sémaphores pour résoudre ces problèmes : les joueurs ne doivent pas être bloqués inutilement pendant leur tour.

Solution : (1 point). Problèmes : si un joueur tire sur une case et qu'un autre joueur déplace un bateau sur cette case ou depuis cette case, ou que deux joueurs tirent sur la même case. Une solution serait d'utiliser un sémaphore pour chaque case initialisé à 1. Dès qu'un joueur sélectionne une case, on bloque la case $P(S_{case}, 1)$, on effectue l'action de tir. Pour un déplacement, double opération : $[P(S_{caseDepart}, 1), P(S_{caseArrivee}, 1)]$.

Problème : le nombre de sémaphores risque d'exploser (largeur $\times$ hauteur cases = nombre de sémaphores). Une autre solution est donc de découper le plateau de jeu en zones et d'utiliser un sémaphore par zone. Pour un tir, on bloque le sémaphore de la zone. Pour un déplacement, si c'est au sein de la même zone, on bloque uniquement le sémaphore de la zone. Si le déplacement impacte deux zones, on bloque les deux zones simultanément : $[P(S_{zoneDepart}, 1), P(S_{zoneArrivee}, 1)]$.

f) Donnez la portion de code permettant au *coordinateur* de créer le TdS (clé IPC définie par la constante `CLE_SEM`) et de l'initialiser. Vous pourrez utiliser des variables ou des constantes pour faciliter l'écriture du code : vous donnerez alors leur valeur et leur signification.

Solution : (1 point). Voici un exemple de code :

```
int nb_zones = ???; /* Le nombre de zones */
int semid = semget((key_t)CLE_SEM, 2 + nb_zones, S_IRUSR | S_IWUSR | IPC_CREAT);

semctl(semid, 0, SETVAL, 1); /* Pour le début */
semctl(semid, 1, SETVAL, 1); /* Pour les tours */
for(i = 0; i < nb_zones; i++)
    semctl(semid, 2 + i, SETVAL, 1); /* Pour chaque zone
...
```

g) Lorsque le *serveur* démarre, il vérifie que le TdS existe. Si ce n'est pas le cas, il se met en attente et vérifie à nouveau, jusqu'à ce que le tableau soit effectivement créé. Donnez la portion de code permettant de réaliser ce test.

Solution : (1 point). Voici l'extrait de code :

```
...
int semid;

while(semid = semget((key_t)CLE_SEM, 0, 0)) {
    printf("Serveur_non_prêt.\n");
    sleep(1);
}
...
```